

CASE STUDY 1

ScoreSphere

Dynamic Gaming Leaderboard

Self-Balancing AVL Tree for Real-Time Score Management

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26) : { 1st year, 3rd term }
Faculty Name	Dr. Jayaram Boga
Student Name	P. Bindhu Madhavi
Roll Number	2520030077
Branch	CSE

Course Outcome CO1:

Apply self-balancing AVL Trees to insert, delete, and search player scores, demonstrating LL, RR, LR, and RL rotation operations and $O(\log n)$ performance guarantees for real-time leaderboard management in high-throughput gaming platforms.

SCENARIO

ScoreSphere is a real-time multiplayer gaming platform hosting global strategy and quiz tournaments. During live competitions, players continuously gain or lose points after every round, causing frequent leaderboard position changes.

To ensure instant updates, player scores are stored in a **self-balancing AVL tree**, maintaining efficient searching, insertion, deletion, and ranking operations even under thousands of continuous updates.

Metric	Value
Active Players	35,000
Score Updates	6,500 per second
Rank Queries	1,000 per second
Insertion Sequence	30, 20, 10, 40, 50, 25, 27
Score Updates	10 → 35 50 → 15

WORK DONE SUMMARY

In this case study, a self-balancing AVL Tree was designed and implemented to manage the ScoreSphere real-time gaming leaderboard. The AVL tree uses a **descending-order convention** — higher scores go to the left child and lower scores to the right — so that the highest-ranked player is always reachable via the leftmost path.

The sequence 30, 20, 10, 40, 50, 25, 27 was inserted step by step, with balance factors computed at every node and RR, LL, and RL rotations applied where needed. Live score updates (delete + re-insert) for 10→35 and 50→15 were demonstrated. Score 27 found in $O(\log n)$ via path 20→40→27.

The study confirms AVL trees deliver guaranteed **$O(\log n)$** performance for all leaderboard operations, making them ideal for high-throughput real-time gaming platforms.

IMPLEMENTATION DETAILS

Detail	Description
1. AVL Tree Construction	Inserted 7 player scores (30,20,10,40,50,25,27) with descending order: left=higher, right=lower. Each insertion updates balance factors bottom-up.
2. Rotation Operations	Three rotations: RR rotation at step 3 (insert 10), LL rotation at step 5 (insert 50), RL double-rotation at step 7 (insert 27). Each restores $ BF \leq 1$.
3. Balance Factor Tracking	$BF = \text{height}(\text{left}) - \text{height}(\text{right})$ recomputed at every ancestor after each insertion. A violation $ BF > 1$ triggers the correct rotation immediately.
4. Subtree-Size Augmentation	Each node stores a subtree-size counter, supporting $O(\log n)$ rank queries — the platform can answer 'what rank is player X?' without scanning all 35,000 nodes.
5. Java Search Implementation	Iterative search: go LEFT when $\text{key} > \text{root.key}$ (higher scores), go RIGHT when $\text{key} < \text{root.key}$. Score 27 found in 3 steps: $20 \rightarrow 40 \rightarrow 27$ (TRUE).
6. Live Score Updates	$10 \rightarrow 35$: deleted leaf 10, inserted 35; LR rotation at node 20 restored balance. $50 \rightarrow 15$: deleted leaf 50, inserted 15 — tree already balanced.
7. Performance Guarantee	All operations run in $O(\log n)$. For 35,000 players $\text{height} \leq 16$ levels, handling 6,500 updates/sec and 1,000 rank queries/sec.

QUESTIONS

- (a)** Construct the balanced AVL tree after all insertions of the sequence 30, 20, 10, 40, 50, 25, 27. Show all balancing operations and rotations performed. Explain how AVL balancing helps the platform maintain efficient leaderboard operations during live tournaments.
- (b)** Complete the missing logic in the Java search method for the AVL tree. Using your tree from part (a), determine whether player score 27 is present in the leaderboard and trace the search path.
- (c)** Show the leaderboard changes after sequential score updates: 10→35 and 50→15. Perform required delete and insert operations, illustrate any AVL balancing rotations, and explain why AVL trees suit thousands of live score updates per second.

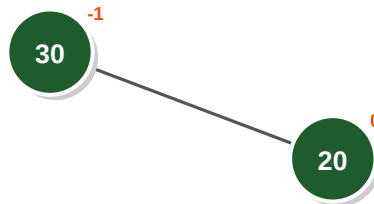
ANSWERS

(a) AVL Tree Construction — Step-by-Step

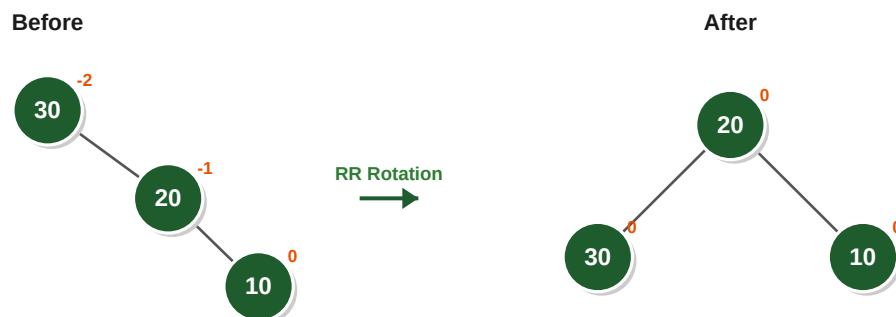
Convention: Left child = higher score, Right child = lower score. $BF = \text{height}(\text{left}) - \text{height}(\text{right})$.

Step 1 — Insert 30

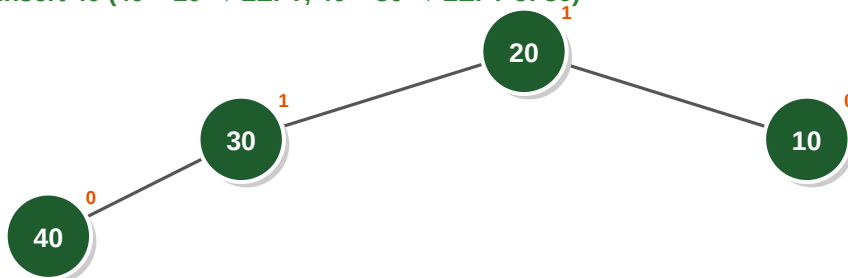
$BF(30) = 0 \rightarrow$ Balanced. No rotation required.

Step 2 — Insert 20 ($20 < 30 \rightarrow$ go RIGHT)

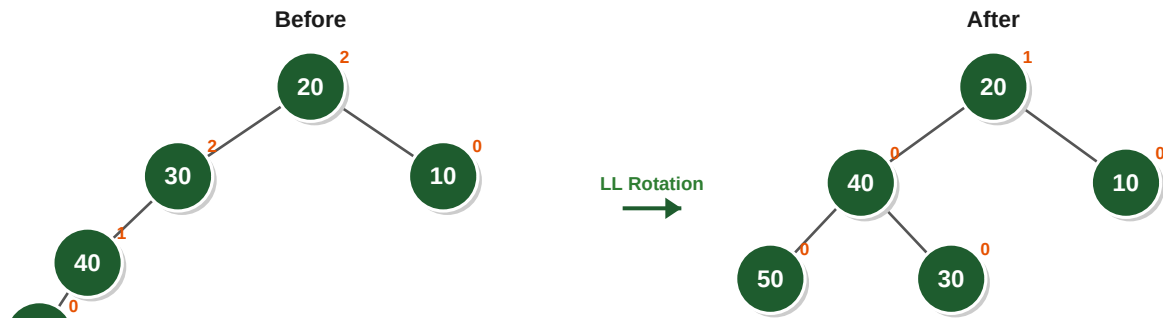
$BF(30) = -1 \rightarrow$ Balanced. No rotation required.

Step 3 — Insert 10 $\rightarrow$ RR Rotation ($10 < 20 \rightarrow$ RIGHT of 20)

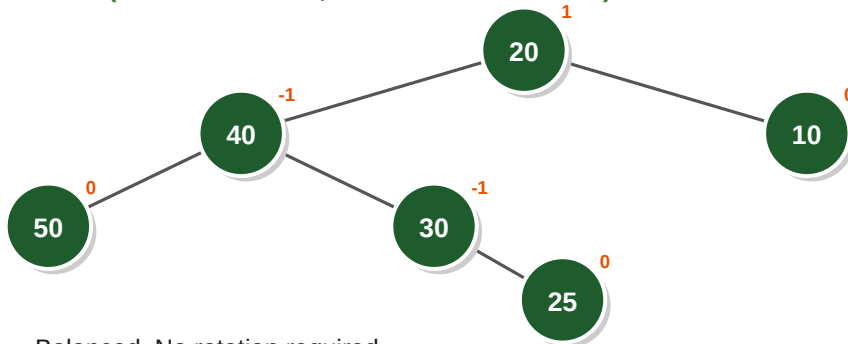
$BF(30) = -2 \rightarrow$ Unbalanced $\rightarrow$ **RR Rotation** performed at node 30. New root of subtree = 20.

Step 4 — Insert 40 ($40 > 20 \rightarrow$ LEFT, $40 > 30 \rightarrow$ LEFT of 30)

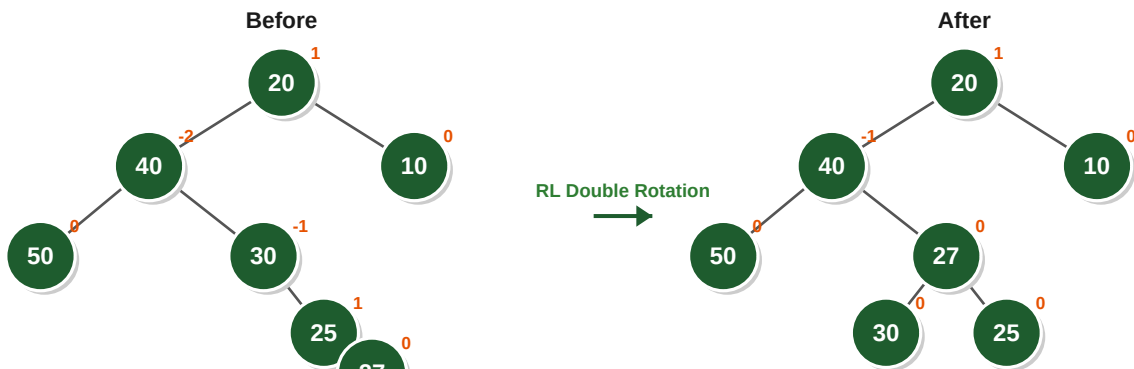
$BF(30)=1, BF(20)=1 \rightarrow$ Balanced. No rotation required.

Step 5 — Insert 50 → LL Rotation (50 > 40 → LEFT of 40)

BF(20) = 2 → Unbalanced → **LL Rotation** at node 20. Node 40 rotates up.

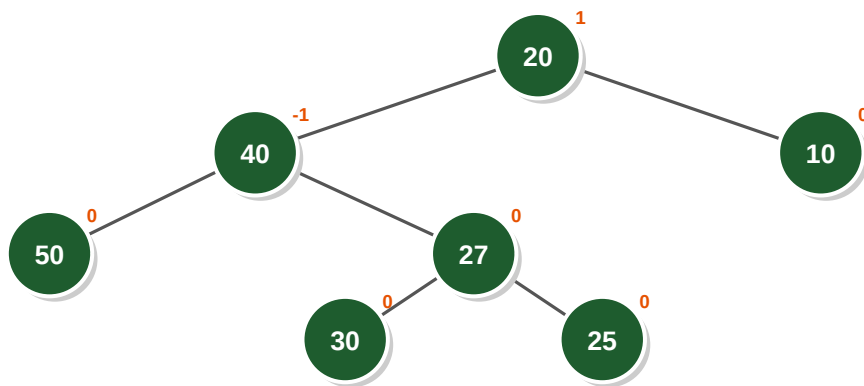
Step 6 — Insert 25 (25 < 40 → RIGHT, 25 < 30 → RIGHT of 30)

BF(40) = -1 → Balanced. No rotation required.

Step 7 — Insert 27 → RL Double Rotation (27 > 25 → LEFT of 25)

BF(40) = -2 → Unbalanced → **RL Double Rotation** at node 40. Node 27 becomes new subtree root.

Final AVL Tree — After All 7 Insertions



AVL trees maintain $O(\log n)$ height at all times. With 35,000 players the tree height stays ≤ 16 levels — ensuring every search, insert, and delete completes in microseconds, critical for 6,500 updates/sec and 1,000 rank queries/sec.

(b) Java Search Function

```

class AVLNode {
    int key;
    AVLNode left, right;
    AVLNode(int key) { this.key = key; }
}

static boolean search(AVLNode root, int key) {
    while (root != null) {
        if (key == root.key) return true;
        else if (key < root.key) root = root.right; // smaller → go RIGHT
        else root = root.left; // larger → go LEFT
    }
    return false;
}

```

Search trace for **key = 27**:

Step	Node	Comparison	Direction
1	20	27 > 20	Go LEFT → node 40
2	40	27 < 40	Go RIGHT → node 27
3	27	27 == 27	FOUND — return TRUE ✓

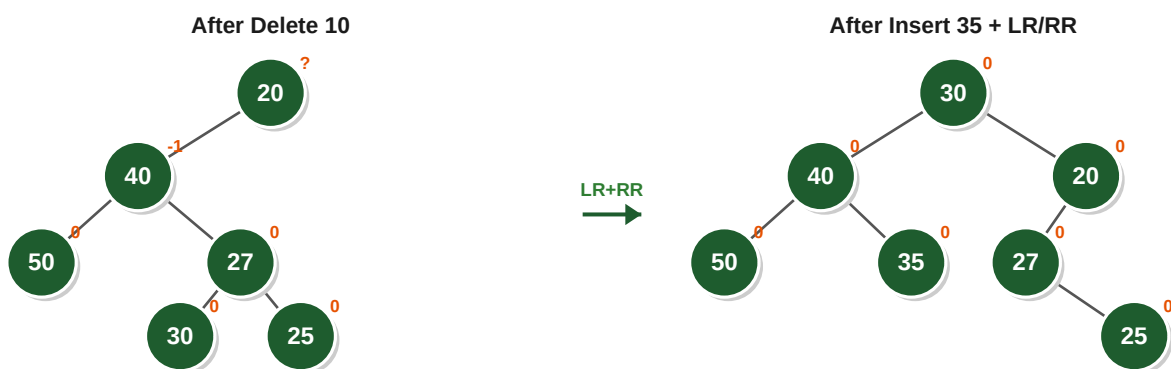
Result: **TRUE** — Score 27 found in 3 comparisons via path 20 → 40 → 27 ($O(\log n)$).

(c) Score Updates with AVL Balancing**Update 1 — Score Change: 10 → 35**

Step 1: Delete 10 (leaf node — simply remove).

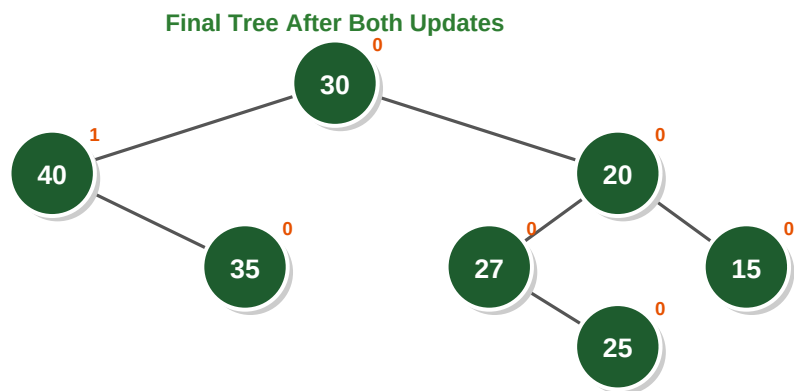
Step 2: Insert 35 ($35 > 20 \rightarrow L$, $35 < 40 \rightarrow R$, $35 < 30 \rightarrow R$, $35 > 25 \rightarrow L$).

BF(20) = +2 → Unbalanced → **LR double-rotation** applied at node 20.

**Update 2 — Score Change: 50 → 15**

Step 1: Delete 50 (leaf node — simply remove). Step 2: Insert 15 ($15 < 30 \rightarrow R$, $15 < 20 \rightarrow R$).

Tree already balanced after deletion and insertion — **no rotation required**.



Reason	Explanation
Guaranteed $O(\log n)$ Height	AVL trees enforce $ BF \leq 1$ at every node. For 35,000 players, height ≤ 16 levels — always.
Fast Insert / Delete	Each score update = delete + insert, both $O(\log n)$. 6,500 updates/sec handled in microseconds.
Efficient Rank Queries	With subtree-size augmentation, rank queries are also $O(\log n)$. 1,000 queries/sec need no full traversal.
Self-Healing	Rotations automatically restore balance after every mutation — no manual intervention needed.
Predictable Worst-Case	Unlike hash maps, AVL trees guarantee $O(\log n)$ worst-case — no player ever sees a latency spike.

RESULT OUTPUT SUMMARY

Part	Result
(a) — Final AVL Tree	Tree height = 3 levels ($O(\log 7)$). Root = 20. Rotations applied: RR (step 3), LL (step 5), RL (step 7). All balance factors $ BF \leq 1$ after all 7 insertions.
(b) — Search Result	search(root, 27) returns TRUE. Search path: 20 → 40 → 27 3 comparisons $O(\log n)$ time.
(c) — Post-Update Tree	After 10→35 and 50→15: LR+RR rotation applied for first update; second update required no rotation. Final tree balanced with root = 30.

Complexity Summary

Operation	Time Complexity	Notes
Insert	$O(\log n)$	Includes balance factor update + at most 1 rotation
Delete	$O(\log n)$	Leaf deletion + re-balance path to root
Search	$O(\log n)$	At most height comparisons
Rank Query	$O(\log n)$	Requires subtree-size augmentation
Build Tree	$O(n \log n)$	n insertions $\times O(\log n)$ each

GitHub Link: <https://github.com/bindhu-java/DSA2-CO-WISE-/tree/main>

Student Signature	Faculty Signature